

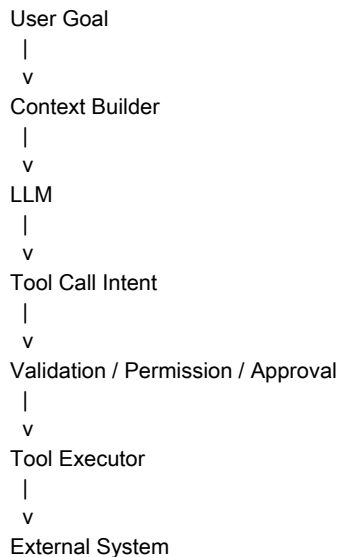
Day05 Part G 学习文档 v1.0 : Tool Result 回流 Runtime (Runtime Feedback Loop)

本文是《从零实现 Agent Runtime》学习阶段的 Day05 Part G 正式学习文档。

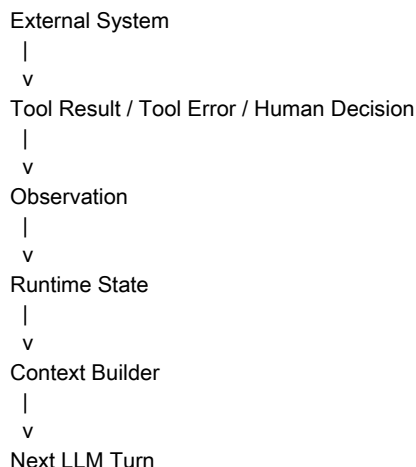
Day05 Part A 建立 Tool Calling 基础模型，Part B 解释 LLM 如何做 Tool Decision，Part C 说明 Tool Schema 如何定义行动契约，Part D 说明 Tool Registry 如何管理能力空间，Part E 说明 Tool Executor 如何把 Tool Call Intent 转成真实执行，Part F 引入 Permission 与 Human Approval 治理层。Part G 开始回答执行后的关键问题：Tool 执行完成后，结果如何回流 Runtime，并影响 Agent 的下一轮思考与行动？

本节定位

Part A-F 已经完成了 Agent Execution Engine 的前半段：



Part G 关注的是执行之后：



本节核心结论是：

Tool Result 不是 Agent 的最终答案，而是 Agent 下一轮推理的输入。Runtime 需要把外部系统返回的结果转成统一的 Observation，写入 Runtime State，再由 Context Builder 选择、压缩、投影给 LLM，从而形成 Agent Feedback Loop。

目录

与 Part E / Part F 的联系

为什么 Agent 需要 Feedback Loop

Tool Result 不等于 Final Answer

Observation 的本质

Observation 与 Message 的区别

为什么不能直接把 Tool Result 塞给 LLM

Observation 数据结构

Observation Source 与 Type

Result Processor 的职责

Observation 如何进入 Runtime State

State Transition 与 Reducer

Observation Store

Context Builder 如何消费 Observation

Observation Selection

Observation Priority

Observation Compression

Observation Eviction

Observation 与 Memory 的边界

Tool Error 也是 Observation

Recovery Strategy

Error Processor

Runtime Policy 与 LLM Reasoning 的边界

Observation 与 Human Approval

Observation 不是简单数据清洗层

Runtime Event Type 与 Domain Event Type

Observation 与 MCP

mini-agent-runtime 设计

工业术语映射

OpenAI Agents SDK / Claude Code / LangGraph 映射

面试视角

下一节学习计划

写书 TODO

写书素材

本 Part 核心认知升级

知识地图

思考题

前置问题回收

与 Part E / Part F 的联系

Part E 说明 Tool Executor 的职责不是：

```
tools[toolCall.name](toolCall.arguments)
```

而是托管完整执行生命周期：

```
Tool Call Intent
|
v
Tool Lookup
|
v
Input Validation
|
v
Permission Check
|
v
Execution
|
v
Result Processing
|
v
Observation
```

Part F 展开了其中的治理层：

```
Permission Check
|
+-- allow
+-- deny
+-- approval_required
```

Part G 展开的是执行结果回流：

```
Result Processing
|
v
Observation
|
v
Runtime State
|
v
Context Builder
```

也就是说：

Part E 关注谁执行，Part F 关注是否允许执行，Part G 关注执行之后如何让 Agent 知道世界发生了什么变化。

为什么 Agent 需要 Feedback Loop

普通聊天模型可以只做一次输入输出：

```
User Message
|
v
LLM
|
v
Assistant Answer
```

但 Agent 不是只回答问题，而是要在外部世界中行动。

一旦 Agent 调用 Tool，就会出现新的事实：

```
Tool Call
|
v
External System
|
v
Result
```

这些事实会影响下一步：

- 是否已经完成用户目标
- 是否需要继续调用 Tool
- 是否需要重试
- 是否需要换一个 Tool
- 是否需要询问用户
- 是否需要把关键结果写入长期 Memory

因此 Agent Loop 不是：

```
Think -> Act -> Done
```

而是：

```
Think -> Act -> Observe -> Think -> Act -> Observe -> ...
```

这就是 ReAct Loop 在 Runtime 中的真实含义。

Tool Result 不等于 Final Answer

Tool Result 是外部系统返回给 Runtime 的结果。

Final Answer 是 Agent 面向用户生成的最终响应。

两者不应该混在一起。

例如用户问：

```
帮我查一下订单 123 的状态
```

LLM 产生 Tool Call Intent：

```
{
  "name": "get_order",
  "arguments": {
    "orderId": "123"
  }
}
```

Tool 返回：

```
{
```

```
"orderId": "123",  
"status": "SHIPPED",  
"estimatedDelivery": "2026-08-06"  
}
```

这不是最终答案。

它只是 Runtime 获得的新事实：

订单 123 已发货，预计 2026-08-06 送达。

Agent 还需要根据这个结果决定：

- 是否直接回复用户
- 是否继续查询物流详情
- 是否发现异常并解释
- 是否把信息整理成自然语言

最终答案可能是：

订单 123 已经发货，预计 2026-08-06 送达。

所以：

Tool Result = World Feedback
Final Answer = User-facing Response

Observation 的本质

Observation 是 Runtime 对外部反馈的统一抽象。

它可以来自：

- Tool 成功结果
- Tool 错误
- Human Approval 决策
- 外部事件
- Timeout
- Cancellation

Observation 不是业务系统的原始返回，也不是直接给 LLM 的聊天消息。

它是：

Runtime 可理解、可存储、可审计、可选择、可投影的反馈事件。

更准确地说：

Observation
= Runtime Event
+ External Feedback
+ Semantic Envelope

它承担三件事：

把外部世界的变化带回 Runtime

让 Runtime State 发生可追踪的状态变化

为下一轮 Context Builder 提供候选上下文

Observation 与 Message 的区别

Message 是对话层对象。

Observation 是 Runtime 层对象。

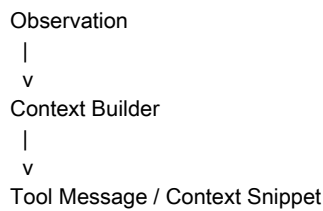
Message 主要描述：

User / Assistant / System / Tool 在对话中说了什么

Observation 主要描述：

Runtime 从外部世界观察到了什么

二者关系可以是：



但 Observation 本身不等于 Message。

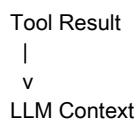
为什么要分开？

- Message 面向模型协议与对话历史
- Observation 面向 Runtime 状态、审计、恢复、压缩和选择
- 同一个 Observation 可以用不同方式投影给不同模型
- 有些 Observation 不应该进入 LLM Context，只应该留在 Runtime State 或 Audit Log

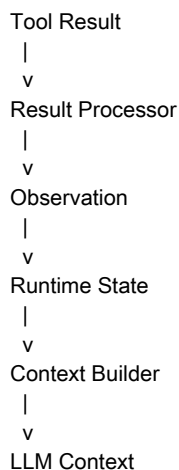
如果把 Observation 直接当 Message 存，会让 Runtime 丧失结构化管理能力。

为什么不能直接把 Tool Result 塞给 LLM

直接把 Tool Result 塞给 LLM 的 Demo 链路是：



工业 Runtime 中更合理的链路是：



原因有五个。

第一，原始 Tool Result 可能太大。

例如搜索结果、日志、文件内容、数据库查询结果可能远远超过 Context Window。

第二，原始 Tool Result 可能包含敏感字段。

例如 token、内部 ID、成本、权限字段、用户隐私字段都不应该直接暴露给模型。

第三，原始 Tool Result 缺少 Runtime 语义。

LLM 需要知道这次反馈是成功、失败、审批拒绝、超时，还是外部事件。

第四，Runtime 需要审计与恢复。

直接塞给 LLM 的文本很难支持 Replay、Resume、Trace 和 Debug。

第五，下一轮 Context 需要选择和压缩。

不是所有 Observation 都同等重要，也不是所有 Observation 都需要被模型看到。

Observation 数据结构

最小 Observation 可以这样设计：

```
type ObservationType =
  | "tool_result"
  | "tool_error"
  | "human_approval"
  | "external_event";

interface Observation {
  id: string;
  type: ObservationType;
  source: ObservationSource;
  status: "success" | "error" | "denied" | "timeout";
  toolCallId?: string;
  data?: unknown;
  error?: ToolError;
  summary: string;
  createdAt: number;
  metadata?: Record<string, unknown>;
}

interface ObservationSource {
  kind: "tool" | "human" | "system" | "external";
  name: string;
}
```

成功 Tool Result：

```
{
  "type": "tool_result",
  "source": {
    "kind": "tool",
    "name": "get_order"
  },
  "status": "success",
  "toolCallId": "call_123",
  "data": {
    "orderId": "123",
    "status": "SHIPPED"
  },
  "summary": "订单 123 已发货。"
}
```

Tool Error :

```
{
  "type": "tool_error",
  "source": {
    "kind": "tool",
    "name": "get_order"
  },
  "status": "error",
  "toolCallId": "call_124",
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "Order 999 was not found.",
    "recoverable": true
  },
  "summary": "订单 999 不存在。"
}
```

Human Approval :

```
{
  "type": "human_approval",
  "source": {
    "kind": "human",
    "name": "approver"
  },
  "status": "denied",
  "toolCallId": "call_125",
  "data": {
    "decision": "deny",
    "reason": "退款金额超过人工审批范围。"
  },
  "summary": "人工审批拒绝执行退款。"
}
```

Observation Source 与 Type

Observation 需要 source，因为 Runtime 要知道反馈从哪里来：

tool / human / system / external

这影响：

- 审计
- 恢复
- 安全策略
- Context 投影
- 事件追踪

Observation 也需要 type，但这个 type 应该是 Runtime 事件类型，而不是业务事件类型。

推荐：

```
type ObservationType =
  | "tool_result"
  | "tool_error"
  | "human_approval"
  | "external_event";
```

不推荐：

```
type ObservationType =
  | "ORDER_SHIPPED"
```



```
| "PAYMENT_FAILED"  
| "CUSTOMER_VIP";
```

原因是 Runtime 不应该被某个业务域污染。

业务含义应该放在 payload 中：

```
{  
  "type": "tool_result",  
  "source": {  
    "kind": "tool",  
    "name": "order_service"  
  },  
  "data": {  
    "domainEvent": "ORDER_SHIPPED",  
    "orderId": "123"  
  }  
}
```

Result Processor 的职责

Result Processor 是 Tool Result 进入 Runtime State 前的一层适配器。

它负责：

- 标准化 Tool Result
- 抽取摘要
- 标记成功或失败
- 识别可恢复错误
- 移除敏感字段
- 记录元数据
- 生成 Observation

链路：

```
Raw Tool Result  
|  
v  
Result Processor  
|  
v  
Observation
```

伪代码：

```
function processToolResult(  
  toolCall: ToolCall,  
  result: ToolResult  
) : Observation {  
  if (result.ok) {  
    return {  
      id: createdAt(),  
      type: "tool_result",  
      source: {  
        kind: "tool",  
        name: toolCall.name  
      },  
      status: "success",  
      toolCallId: toolCall.id,  
      data: sanitize(result.data),  
      summary: summarize(result.data),  
    }  
  }  
}
```

```

        createdAt: Date.now()
    };
}

return {
    id: createId(),
    type: "tool_error",
    source: {
        kind: "tool",
        name: toolCall.name
    },
    status: "error",
    toolCallId: toolCall.id,
    error: normalizeError(result.error),
    summary: summarizeError(result.error),
    createdAt: Date.now()
};
}

```

Result Processor 不是业务逻辑中心。

它不应该决定订单是否应该退款，也不应该决定用户是否是 VIP。

它只负责把业务反馈翻译成 Runtime 能处理的 Observation。

Observation 如何进入 Runtime State

Observation 不应该只是函数里的临时变量。

它应该进入 Runtime State :

```

interface RuntimeState {
    sessionId: string;
    messages: Message[];
    toolCalls: ToolCallRecord[];
    observations: Observation[];
    approvals: ApprovalRecord[];
    status: RuntimeStatus;
}

```

因为 Runtime 需要 :

- 后续推理可见
- 异常恢复可用
- 审计可追踪
- Context Builder 可选择
- Memory Evaluator 可评估

如果 Observation 不进 State , Agent Loop 就会断 :

```

Tool Result
|
v
lost

```

正确链路是 :

```

Tool Result
|
v
Observation
|
v

```

Runtime State
|
v
Next Turn

State Transition 与 Reducer

Observation 进入 State，本质是一次 State Transition。

不是：

```
state.observations.push(observation);
```

而是：

Event
|
v
Reducer
|
v
New State

示例：

```
type RuntimeEvent =  
  | { type: "tool_completed"; toolCall: ToolCall; result: ToolResult }  
  | { type: "tool_failed"; toolCall: ToolCall; error: ToolError }  
  | { type: "approval_resolved"; approval: ApprovalRecord };  
  
function runtimeReducer(  
  state: RuntimeState,  
  event: RuntimeEvent  
): RuntimeState {  
  switch (event.type) {  
    case "tool_completed": {  
      const observation = processToolResult(event.toolCall, {  
        ok: true,  
        data: event.result  
      });  
  
      return {  
        ...state,  
        observations: [...state.observations, observation],  
        status: "thinking"  
      };  
    }  
  
    case "tool_failed": {  
      const observation = processToolResult(event.toolCall, {  
        ok: false,  
        error: event.error  
      });  
  
      return {  
        ...state,  
        observations: [...state.observations, observation],  
        status: event.error.recoverable ? "thinking" : "failed"  
      };  
    }  
  }  
}
```

这样做的价值是：

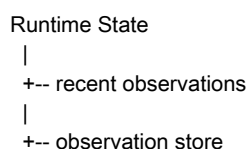
- 可追踪：知道每个 Observation 从哪个 Event 来

- 可恢复：失败后可以 Replay Event 重建 State
- 可审计：每次状态变化都能记录
- 可测试：Reducer 可以单独测试

Observation Store

随着 Agent Loop 变长，Observation 数量会增加。

Runtime 可以把 Observation 分为两层：



Recent observations 用于当前会话短期上下文。

Observation Store 用于：

- 历史审计
- 长任务恢复
- Trace 查看
- Debug
- Memory 写入评估

Observation Store 不是 Memory。

它记录的是 Runtime 观察到的执行反馈，而 Memory 记录的是对未来任务有长期价值的信息。

Context Builder 如何消费 Observation

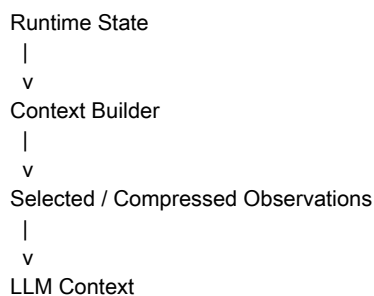
Day04 已经建立一个核心认知：

Runtime State 不等于 LLM Context。

Part G 延续这个原则：

Observation 不会自动全部进入 LLM Context。

正确链路：



Context Builder 对 Observation 做四件事：

- Selection：选择哪些 Observation 相关
- Priority：判断哪些 Observation 更重要

Compression : 压缩长结果

Eviction : 丢弃或降级不再重要的结果

Observation Selection

Observation Selection 解决 :

当前这一轮 LLM 需要看到哪些反馈 ?

选择依据包括 :

- 是否来自最近的 Tool Call
- 是否和当前 Goal 相关
- 是否是错误或审批结果
- 是否影响下一步行动
- 是否被后续 Observation 依赖

示例 :

```
function selectObservations(
  state: RuntimeState,
  goal: UserGoal
): Observation[] {
  return state.observations.filter((observation) => {
    return (
      isRecent(observation) ||
      isRelevantToGoal(observation, goal) ||
      isError(observation) ||
      isApprovalDecision(observation)
    );
  });
}
```

Observation Priority

Observation Priority 决定排序和预算分配。

常见优先级因素 :

Recency
Relevance
Importance
Dependency

Recency : 越近越重要。

Relevance : 越贴近当前任务越重要。

Importance : 错误、审批、最终状态通常比中间日志重要。

Dependency : 后续动作依赖的 Observation 必须保留。

例如 :

Tool A 查询订单成功
Tool B 查询物流成功
Tool C 发送通知失败

下一轮最需要看到的可能不是订单详情 , 而是 :

发送通知失败，原因是邮箱地址无效。

Observation Compression

Observation Compression 解决：

结果太长时如何给模型看？

原始结果：

```
{
  "items": [
    { "title": "...", "content": "very long..." },
    { "title": "...", "content": "very long..." }
  ]
}
```

投影给 LLM 的内容应该是：

搜索返回 20 条结果，最相关的 3 条是：

1. ...
2. ...
3. ...

Compression 不是随便截断。

它应该保留：

- 决策相关字段
- 错误原因
- 资源 ID
- 用户可见结论
- 下一步行动需要的信息

Observation Eviction

Observation Eviction 解决：

旧的 Observation 何时不再进入当前 Context？

可以有三层：

```
recent window
summary
archive / store
```

最近窗口保留完整 Observation。

Summary 保留压缩后的结论。

Archive / Store 保留审计与恢复数据，但不再默认投影给 LLM。

注意：

Eviction 是从 Context 中淘汰，不是从系统中删除。

Observation 与 Memory 的边界

Observation 是 Runtime 观察到的事件。

Memory 是对未来任务有长期价值的信息。

Observation :

订单 123 查询结果：已发货。

通常只属于当前任务。

Memory :

用户偏好使用 pnpm。

可能影响未来任务。

关系：

```
Observation
|
v
Memory Evaluator
|
+-- write to memory
+-- ignore
```

不是所有 Observation 都应该进入 Memory。

Day06 会继续展开：

Memory 是不是 Observation 的长期存储？

更准确的答案是：

Memory 可以从 Observation 中抽取，但 Memory 不是 Observation Store。

Tool Error 也是 Observation

Demo Agent 常见错误处理：

```
try {
  const result = await tool.execute(args);
  return result;
} catch (error) {
  throw error;
}
```

这会让 Agent Loop 中断。

工业 Runtime 应该把错误变成 Observation：

```
Tool Error
|
v
Error Observation
|
v
Runtime State
|
v
Next LLM Turn / Recovery Policy
```

因为错误也是外部世界反馈。

例如：

```
{
  "type": "tool_error",
  "status": "error",
}
```

```
"error": {
  "code": "RATE_LIMITED",
  "message": "Too many requests",
  "recoverable": true,
  "retryAfterMs": 3000
},
"summary": "调用搜索工具时触发限流，可在 3 秒后重试。"
}
```

下一轮 Agent 可以据此选择：

- 等待后重试
- 换一个 Tool
- 降级回复
- 询问用户
- 终止任务并解释

Recovery Strategy

常见恢复策略有四类。

Retry：

同一个 Tool，同一类参数，稍后再试。

适合：

- 网络抖动
- 限流
- 临时服务不可用

Fallback：

使用降级 Tool 或缓存数据。

适合：

- 主服务不可用
- 搜索源不可用
- 模型能力降级

Alternative Action：

换一种行动路径。

例如查询订单失败后，先查询用户最近订单列表。

Ask User：

缺少必要信息或歧义无法自动恢复时，询问用户。

例如：

我没有找到订单 123。你能确认一下订单号，或者提供下单手机号吗？

Error Processor

Error Processor 是 Result Processor 的错误分支。

它负责：

- 归一化错误类型
- 判断是否可恢复
- 提取 retryAfter / rateLimit / validation details
- 隐藏内部错误细节
- 生成面向 LLM 的摘要
- 生成面向审计的结构化记录

示例：

```
interface ToolError {
  code: string;
  message: string;
  recoverable: boolean;
  retryAfterMs?: number;
  details?: unknown;
}

interface ErrorObservation extends Observation {
  type: "tool_error";
  error: ToolError;
}
```

内部错误：

ECONNRESET at 10.2.3.4:5432

不应该直接给 LLM。

可以转成：

订单服务暂时不可用，这是一个可重试错误。

Runtime Policy 与 LLM Reasoning 的边界

错误恢复不应该完全交给 Tool，也不应该完全交给 LLM。

Tool 知道局部错误：

这个 API 调用为什么失败。

LLM 知道目标语义：

用户真正想完成什么。

Runtime Policy 知道系统边界：

最多重试几次，哪些错误可重试，哪些动作必须停止。

因此工业 Agent 更合理的模式是：

```
Runtime Policy
|
v
Recovery Options
|
v
LLM Reasoning
|
v
Next Action
```

例如：

Runtime: 这个错误最多允许重试 2 次。

LLM: 当前任务仍需要这个结果，因此选择重试。

Runtime: 执行重试并记录 Observation。

Observation 与 Human Approval

Part F 中 Human Approval 不是普通权限本身，而是 Runtime Event。

Part G 补齐它的回流：

```
approval_required
|
v
Suspend
|
v
Human Decision
|
v
Approval Observation
|
v
Runtime State
|
v
Resume / Stop / Next LLM Turn
```

审批通过：

```
{
  "type": "human_approval",
  "status": "success",
  "data": {
    "decision": "approve"
  },
  "summary": "人工已批准执行退款。"
}
```

审批拒绝：

```
{
  "type": "human_approval",
  "status": "denied",
  "data": {
    "decision": "deny",
    "reason": "金额过高"
  },
  "summary": "人工拒绝执行退款，原因是金额过高。"
}
```

审批超时：

```
{
  "type": "human_approval",
  "status": "timeout",
  "summary": "人工审批超时，Runtime 停止执行该动作。"
}
```

审批结果进入 State 后，Agent 才能继续合理行动。

Observation 不是简单数据清洗层

学习记录中有一个重要追问：

Observation 整体感受下来更像是一层数据清洗格式化层？如果 Observation type 定义成固定枚举，会不会导致 Tool 结果必须按 Runtime 内置的类型格式返回？这是不是变相把业务逻辑写进 Runtime？

这个问题触及 Agent Runtime 的核心边界。

Observation 确实包含数据清洗与格式化，但它不只是清洗层。

更准确地说：

- Observation
 - = Data Normalization
 - + Runtime Semantics
 - + Event Boundary
 - + Context Candidate
 - + Audit Record

它至少承担四层职责：

数据标准化：让不同 Tool 的输出进入统一结构

Runtime 语义：告诉 Runtime 这是成功、失败、审批、超时还是外部事件

状态驱动：作为 State Transition 的输入

上下文候选：等待 Context Builder 决定是否投影给 LLM

所以 Observation 是 Runtime 与业务世界之间的隔离层。

Runtime Event Type 与 Domain Event Type

关键原则：

Runtime Event Type 要稳定、通用；Domain Event Type 要留在 payload 中。

错误设计：

```
type ObservationType =  
  | "ORDER_SHIPPED"  
  | "PAYMENT_FAILED"  
  | "CUSTOMER_REFUND_CREATED";
```

这样 Runtime 会被电商业务污染。

正确设计：

```
type ObservationType =  
  | "tool_result"  
  | "tool_error"  
  | "human_approval"  
  | "external_event";
```

业务信息放到 data：

```
{  
  "type": "tool_result",  
  "source": {  
    "kind": "tool",  
    "name": "order_service"  
  },  
  "data": {  
    "event": "ORDER_SHIPPED",  
    "orderId": "123",  
    "status": "SHIPPED"  
  }  
}
```

```
    },  
    "summary": "订单 123 已发货。"  
  }  
}
```

Runtime 只关心：

这是一次 Tool 成功反馈。

业务 Agent 或 Tool Adapter 关心：

订单已经发货。

这就是 Runtime Layer 与 Domain Layer 的边界。

Observation 与 MCP

MCP 提供外部能力连接：

```
MCP Server  
|  
v  
Tool Schema / Tool Call / Tool Result
```

但 MCP 不应该进入 Runtime Core。

无论 Tool 来自：

- 本地函数
- HTTP API
- MCP Server
- Plugin

最终都应该转换成 Runtime 统一理解的 Observation：

```
Internal Tool  
|  
v  
Observation  
  
MCP Tool  
|  
v  
Observation
```

也就是说：

MCP belongs to Tool Layer.
Observation belongs to Runtime Layer.

这能保证 Runtime Core 不绑定某一种外部工具协议。

mini-agent-runtime 设计

最小实现可以分成六个模块。

1. ToolResult

```
type ToolResult<T = unknown> =  
| {  
  ok: true;  
  data: T;  
  metadata?: Record<string, unknown>;  
}
```

```

    }
  | {
    ok: false;
    error: ToolError;
    metadata?: Record<string, unknown>;
  };

```

2. Observation

```

type ObservationType =
  | "tool_result"
  | "tool_error"
  | "human_approval"
  | "external_event";

interface Observation {
  id: string;
  type: ObservationType;
  source: {
    kind: "tool" | "human" | "system" | "external";
    name: string;
  };
  status: "success" | "error" | "denied" | "timeout";
  toolCallId?: string;
  data?: unknown;
  error?: ToolError;
  summary: string;
  createdAt: number;
  visibleToModel?: boolean;
  metadata?: Record<string, unknown>;
}

```

3. ResultProcessor

```

interface ResultProcessor {
  process(toolCall: ToolCall, result: ToolResult): Observation;
}

```

4. RuntimeState

```

interface RuntimeState {
  sessionId: string;
  messages: Message[];
  toolCalls: ToolCallRecord[];
  observations: Observation[];
  status: "idle" | "thinking" | "executing" | "waiting_approval" | "failed";
}

```

5. RuntimeReducer

```

function applyObservation(
  state: RuntimeState,
  observation: Observation
): RuntimeState {
  return {
    ...state,
    observations: [...state.observations, observation],
    status: nextStatus(state, observation)
  };
}

```

6. ContextBuilder

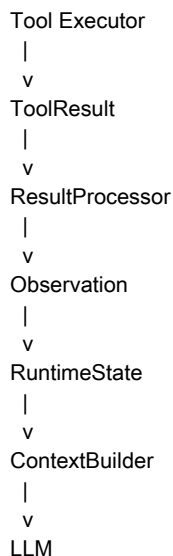
```

interface ObservationProjection {
  role: "tool";
  content: string;
  observationId: string;
}

```

```
function projectObservations(
  observations: Observation[]
): ObservationProjection[] {
  return observations
    .filter((observation) => observation.visibleToModel !== false)
    .map((observation) => ({
      role: "tool",
      observationId: observation.id,
      content: observation.summary
    }));
}
```

最小闭环：



工业术语映射

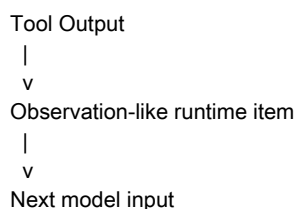
| 学习术语 | 工业术语 | 含义 || --- | --- | --- || Tool Result | Tool Output / Function Result | Tool 执行后返回的原始结果 || Observation | Observation / Runtime Event | Runtime 对外部反馈的统一抽象 || Result Processor | Result Adapter / Output Processor | 把原始结果转换成 Observation || Error Observation | Error Event / Failure Observation | 可进入 Agent Loop 的错误反馈 || Runtime State Feedback | State Update / State Transition | Observation 驱动 Runtime State 更新 || Observation Store | Event Log / Trace Store | 保存执行反馈和审计记录 || Context Projection | Context Rendering | 把 State 中的信息投影给 LLM || Recovery Strategy | Error Recovery Policy | 错误后的恢复策略 || Feedback Loop | Agent Loop / ReAct Loop | 思考、行动、观察、再思考 |

OpenAI Agents SDK / Claude Code / LangGraph 映射

OpenAI Agents SDK

OpenAI Agents SDK 中，Tool 调用结果会回到下一轮模型上下文。

从 Runtime 视角看：



开发者未必直接操作 Observation 类，但框架内部需要处理同类问题：

- Tool output 如何表示
- Tool error 如何回流
- 下一轮模型看到什么
- Trace 中如何记录 Tool 执行

Claude Code

Claude Code 的核心体验来自持续反馈：

```
Read File
|
v
Observation
|
v
Edit File
|
v
Observation
|
v
Run Test
|
v
Observation
|
v
Fix
```

文件读取结果、命令输出、测试失败、权限请求、用户确认，本质上都要被 Runtime 组织成下一轮可用上下文。

LangGraph

LangGraph 显式强调 State 与节点执行。

映射关系：

```
Tool Node Output
|
v
State Update
|
v
Next Node
```

Part G 的 Observation 可以理解为 LangGraph State Update 中的一类结构化事件。

面试视角

Q1：Tool Result 和 Final Answer 有什么区别？

Tool Result 是外部系统返回给 Runtime 的执行反馈，Final Answer 是 Agent 面向用户生成的最终自然语言响应。Tool Result 通常要先变成 Observation，写入 Runtime State，再由 Context Builder 投影给 LLM，最后由 LLM 判断是否回复用户或继续行动。

Q2：为什么需要 Observation？

因为 Runtime 不能直接依赖各种 Tool 的原始返回格式。Observation 把 Tool 成功、Tool 错误、Human Approval、Timeout 等反馈统一成 Runtime 可存储、可审计、可恢复、可选择、可投影的事件。

Q3：为什么 Error 也要变成 Observation？

错误也是外部世界反馈。把 Error 变成 Observation 后，Agent 可以重试、降级、换路径、询问用户或终止任务，而不是让异常直接打断 Agent Loop。

Q4：Observation 和 Memory 有什么区别？

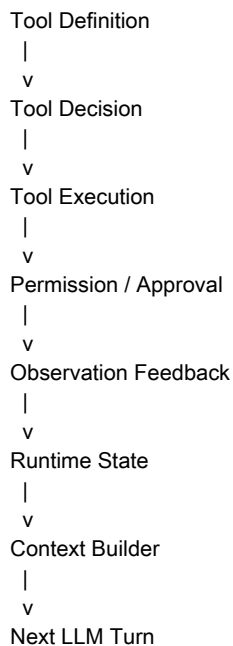
Observation 是当前任务中 Runtime 观察到的事件，Memory 是对未来任务有长期价值的信息。Memory 可以从 Observation 中抽取，但 Observation Store 不等于 Memory。

Q5：Observation type 固定枚举会不会把业务逻辑写进 Runtime？

不会，前提是枚举是 Runtime 事件类型，而不是业务事件类型。Runtime type 应该保持通用，如 tool_result、tool_error、human_approval；业务含义留在 data 或 Tool Adapter 中。

下一节学习计划

Day05 Execution Engine 到 Part G 已经形成闭环：



下一阶段进入：

Day06：Memory System

核心问题：

当 Agent 已经可以行动并观察结果后，哪些信息应该被长期记住？

Day06 计划：

Memory 基础模型

Conversation Memory

Semantic Memory

Memory Retrieval

Memory Write Policy

写书 TODO

Chapter : Agent Execution Engine

需要补充以下章节素材。

Tool Calling 不是 Function Calling

Tool Registry 是 Capability Management Center

Tool Executor 是 Runtime Managed Execution Kernel

Permission Layer 是 Agent 的行动治理边界

Observation Model 是 Agent Feedback Loop 的核心

Error Recovery 让 Agent 从一次性脚本变成可恢复系统

Context Builder 决定 Tool Result 如何影响下一轮推理

Part G 可作为 Execution Engine 的收束章节：

一个 Agent 不是因为能调用 Tool 才成为 Agent，而是因为它能根据行动后的反馈继续调整自己的行为。

写书素材

素材 1：普通程序 vs Agent Runtime

普通程序：

input -> function -> output

Agent Runtime：

goal -> reason -> act -> observe -> update state -> reason again

区别不在于有没有函数调用，而在于有没有反馈闭环。

素材 2：Observation 抽象

Observation 是 Runtime 对外部世界的统一观察。

它不是业务对象，也不是聊天消息，而是介于两者之间的 Runtime Event。

素材 3：Runtime 越来越像 Workflow Engine

当 Runtime 支持：

- State
- Event
- Reducer
- Suspend / Resume
- Approval
- Retry
- Error Recovery
- Trace

它就不再是简单的 LLM Wrapper，而是在演化成 Agent Workflow Engine。

素材 4：架构边界

Observation Layer 和 Domain Layer 必须分离。

Runtime 负责通用执行语义：

success / error / approval / timeout

业务 Tool 负责领域语义：

order shipped / payment failed / file changed

本 Part 核心认知升级

升级 1

从：

Tool Result 是调用结果

升级为：

Tool Result 是下一轮推理的输入

升级 2

从：

把 Tool Result 塞回 LLM

升级为：

Tool Result -> Observation -> Runtime State -> Context Builder -> LLM

升级 3

从：

Error 是异常

升级为：

Error 是 Agent 可利用的反馈

升级 4

从：

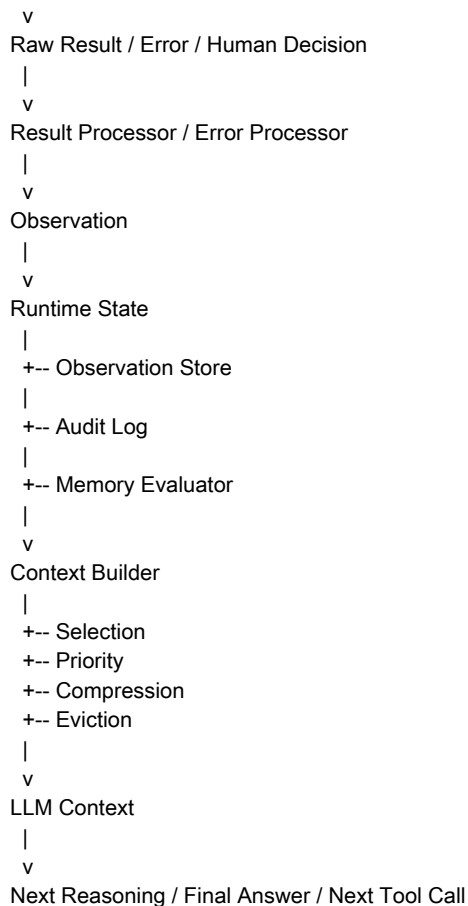
Observation 是数据清洗格式

升级为：

Observation 是 Runtime 与业务世界之间的隔离层

知识地图

```
Tool Call Intent
|
v
Tool Executor
|
v
External System
|
```



思考题

- 为什么 Tool Result 不应该直接等同于 Final Answer ?
- Observation 和 Message 的边界在哪里 ?
- 为什么 Error Observation 能提升 Agent 的可靠性 ?
- Context Builder 在 Observation 回流中承担什么职责 ?
- Observation Store 和 Memory System 有什么区别 ?
- Runtime Event Type 和 Domain Event Type 为什么要分离 ?
- MCP Tool Result 为什么最终也应该进入统一 Observation ?

前置问题回收

Q1 : Tool Result 是否可以直接给用户 ?

可以在简单场景中直接转换成用户回答，但在 Runtime 设计上不应该把 Tool Result 当成 Final Answer。更稳妥的链路是先形成 Observation，再由 LLM 或 Response Builder 生成最终回复。

Q2 : Observation 是不是数据清洗格式化层 ?

它包含数据清洗，但不止于清洗。Observation 还有 Runtime 语义、事件边界、状态驱动、审计与上下文候选职责。

Q3 : 固定 Observation type 会不会限制业务 Tool ?

不会。固定的是 Runtime 通用事件类型，业务语义应该放在 payload、source、metadata 或 Tool Adapter 中。

Q4：什么时候 Observation 应该进入 Memory？

当 Observation 中包含对未来任务有长期价值的信息时，才由 Memory Evaluator 抽取并写入 Memory。大多数执行反馈只属于当前任务和审计记录。

参考

- ChatGPT 分享记录：<https://chatgpt.com/share/6a71a752-1480-83e8-b1f1-fd0632e17d35>
- 本地源记录：source/day05-part-g-chatgpt-share-source.md